Highlights

Structure-Guided Neural Mask Knowledge Editing for Large Language Models via Graph-Aware Attribution

Ruinan Liu*, Fulong Lin, Qingyu Wang, Xin Wang, Zhihua Fu

- SNKE links neuron attribution with target residual directions and knowledge graph structure.
- Edit-centered subgraphs refine neuron selection before parameter updates.
- Soft-gated neuron updates improve edit efficacy and prompt generality.
- Experiments show stronger multi-hop portability and long-range editing stability.

Structure-Guided Neural Mask Knowledge Editing for Large Language Models via Graph-Aware Attribution

Ruinan Liu*[a], Fulong Lin[a,b], Qingyu Wang[c], Xin Wang[a], Zhihua Fu[a]

[a]*Hangzhou Institute of Technology, Xidian University, Hangzhou, 311231, Zhejiang, China*
[b]*School of Mathematics and Statistics, Nanjing University of Information Science and Technology, Nanjing, 210044, Jiangsu, China*
[c]*Wuhan Sanjiang Aerospace Network Communication Co., Ltd., Wuhan, Hubei, China*

Abstract

Large language models store substantial factual knowledge in their parameters, yet post-training factual changes are difficult to incorporate without disturbing unrelated knowledge. Neuron-level editors usually select neurons by activation or gradient strength, but a strong response does not guarantee that the resulting update moves the representation toward the edited fact. This problem becomes more visible under paraphrased prompts, neighboring facts, multi-hop questions, and long edit sequences. We propose Structure-Guided Neural Mask Knowledge Editing (SNKE), a graph-aware neuron-level editing framework that links neuron attribution to target update directions and local relational structure. SNKE estimates a target residual direction, refines neuron attribution with an edit-centered knowledge subgraph, and applies the update through a soft gate. Across multiple benchmarks and backbone models, SNKE improves edit efficacy and generality, with stronger multi-hop portability and long-range editing stability. These results suggest that reliable editing requires sparse parameter changes guided by both target direction and relational structure.

Keywords: Knowledge editing, Large language models, Knowledge graph, Neuron attribution

1. Introduction

Large language models (LLMs) perform well in language understanding, generation, question answering, and reasoning [1, 2]. Their parameters also store a wide range of factual associations [3, 4]. Once training ends, however, these associations can become outdated, incorrect, or biased. Recent surveys on continual learning [5] and knowledge maintenance [6] identify temporal knowledge drift and post-training updates as major obstacles to deploying LLMs in changing environments. Entity attributes, organizational relations, and time-sensitive events may change after training, while full fine-tuning is often too costly and too broad for localized factual correction. This motivates knowledge editing methods that revise specific facts without retraining the whole model.

In response to these challenges, knowledge editing has become an active research direction [7, 8]. Its core objective is to enable the model to generate the updated target fact correctly by modifying only a limited set of parameters associated with the target facts while preserving the model's original capabilities and maintaining the stability of most of its knowledge. Existing research has proposed various knowledge editing methods, including ROME [9], MEMIT [10], MEND [11], and SERAC [12]. Among these, methods based on locating and modifying specific parameters have been proven effective for achieving local knowledge updates. Furthermore, the neuron-level knowledge editing method NMKE [13] identifies important neurons related to the target knowledge through attribution analysis and utilizes a sparse masking mechanism to implement local parameter updates, improving editing efficiency and locality in prior evaluations.

The key challenge is that neuron response and edit usefulness are not equivalent. A neuron may be strongly activated by an edit prompt or receive a large gradient, but its update can still move the hidden representation away from the target fact. This mismatch is difficult to detect in single-prompt evaluation. It becomes more visible when the same edit is tested with paraphrased prompts, neighboring facts, multi-hop reasoning chains, or long edit sequences. Therefore, effective knowledge editing requires identifying active neurons and judging whether their update directions are compatible with the target factual change. As a result, the model may successfully write the new knowledge but still interfere with adjacent facts, reducing local consistency. This issue becomes particularly pronounced in multi-fact editing, sequential editing, or complex knowledge-association scenarios [14, 15].

On the other hand, existing knowledge editing methods generally overlook the inherent structural characteristics of real-world knowledge. In fact, factual knowledge can often be represented as knowledge graphs, where entities are connected by typed relations. Existing methods primarily rely on prompt-driven internal attribution signals but lack modeling of structural dependencies among knowledge items, making it difficult to accurately identify the neurons that are truly relevant to the

Email addresses: liurn2026@gmail.com (Ruinan Liu*), 202213870038@nuist.edu.cn (Fulong Lin), qingyuwang.lll@gmail.com (Qingyu Wang), wangxin03@xidian.edu.cn (Xin Wang), fuzhihua@xidian.edu.cn (Zhihua Fu)

target fact. Particularly when knowledge involves multi-hop associations or semantic coupling, relying solely on text-based attribution often leads to the diffusion of the editing scope, further reducing model stability.

Traditional methods typically rely solely on prompt inputs to drive neuron attribution and generate hard masks for parameter updates based on importance scores. In contrast, our method builds upon this foundation by introducing target residual direction constraints and knowledge graph structural information. Through a structure-guided attribution reweighting mechanism, we achieve more precise neuron selection and employ a soft gating update strategy to enhance the stability of knowledge editing.

We propose Structure-Guided Neural Mask Knowledge Editing (SNKE), a graph-aware neuron-level editing framework that links neuron attribution to both target update directions and relational structure. Instead of selecting neurons only by their response strength, SNKE evaluates whether candidate updates move the hidden representation toward the edited fact, refines this attribution with an edit-centered knowledge subgraph, and applies the resulting update through a soft gate to reduce unstable parameter changes. Specifically, SNKE first optimizes a target representation and derives a target residual vector. The residual is used to measure whether neuron update directions are aligned with the target knowledge direction. SNKE then constructs an edit-centered knowledge subgraph and encodes its local relational structure with a Relational Graph Convolutional Network (RGCN). The encoded structural signal reweights neuron attribution scores through lightweight residual modulation. Finally, a soft-gated update mechanism replaces hard masking, reducing interference while preserving effective update directions.

The main contributions of this paper are summarized as follows:

(1) We propose SNKE, a structure-guided neuron-level knowledge editing framework that treats neuron attribution as a joint target-direction and relational-structure selection problem.

(2) We design a target-residual-aligned attribution mechanism that selects edit-relevant neurons by measuring whether their update directions support the target factual change.

(3) We introduce graph-guided attribution reweighting and soft-gated parameter updates to reduce unstable neuron selection while preserving effective update directions.

(4) We provide a systematic evaluation of SNKE under direct editing, multi-hop portability, computational cost, neuron modification patterns, and long-range sequential editing, revealing both its strengths and its locality-memory trade-offs.

2. Related Work

2.1. Large Language Model Knowledge Editing

Large language model knowledge editing aims to update specific factual knowledge stored in a model without retraining the entire model while preserving the model's original capabilities. Early research primarily relied on fine-tuning to achieve knowledge updates, but such methods typically suffer from high computational costs, poor locality, and a tendency to cause catastrophic forgetting. Recent surveys and empirical studies have summarized LLM knowledge editing methods from different perspectives, including external-memory editing, parameter-preserving editors, optimization-based editors, and locate-then-edit methods [16, 17, 18]. These studies also emphasize that edit success alone is insufficient; generality, locality, portability, and preservation of the model's original capabilities must also be considered.

To improve the accuracy and efficiency of knowledge editing, researchers have proposed a series of localized parameter editing methods. ROME [9] was the first to propose locating knowledge storage within specific feed-forward network (FFN) layers in a Transformer and modifying key parameters via rank-one updates to achieve single-fact knowledge editing. Building on this, MEMIT [10] extends ROME to batch knowledge editing scenarios, enabling large-scale fact modification through multi-layer joint updates. MEND [11] uses a learned auxiliary network to transform gradients into model updates. SERAC [12] instead relies on an external memory mechanism to support semi-parametric editing and retrieval.

Although the aforementioned methods effectively improve the success rate of knowledge editing, their update granularity remains coarse, typically acting directly on the weight matrix, which makes it difficult to achieve finer-grained knowledge localization. Consequently, an increasing number of studies are focusing on neuron-level knowledge editing methods.

2.2. Neuron-Level Knowledge Localization and Attribution

Previous research has shown that knowledge in large language models is often distributed across specific layers and neurons [19]. Based on this observation, neuron-level knowledge editing methods use attribution analysis to identify key neurons associated with target knowledge, thereby enabling sparser and more localized parameter updates.

Knowledge Neurons [20] identified key neurons associated with specific factual knowledge through gradient attribution analysis. Subsequently, NMKE [13] built upon this foundation to propose a neural masking knowledge editing framework. By employing multi-prompt attribution analysis and dynamic mask generation, it achieves selective updates of important neurons, with the goal of reducing the scope of editing and improving local stability.

However, most existing neuron attribution methods rely on activation values or gradient signals, which can only indicate whether a neuron responds to the input but cannot measure whether the direction of neuron updates truly drives the model's hidden state toward the target knowledge. Recent multilingual neuron-level editing studies also suggest that some factual neurons can be language-agnostic, while unconstrained editing may still introduce cross-lingual conflicts or multi-hop inconsistency [21, 22]. Furthermore, most methods rely solely on text prompts to drive attribution, ignoring the inherent structural relationships between pieces of knowledge. This can lead to situations where the model successfully writes the target

knowledge but inadvertently interferes with adjacent knowledge, thereby reducing the stability of knowledge editing.

These limitations call for attribution methods that evaluate neuron sensitivity to prompts, along with the direction of the induced factual change.

These studies show that neuron-level editing can reduce the scope of parameter modification, but they also reveal a remaining gap. Most attribution scores identify neurons that are sensitive to an input, whereas fewer methods examine whether the corresponding update direction supports the intended factual change. This gap becomes especially important when evaluation moves beyond direct rewrite success to generality, locality, and sequential editing stability.

2.3. Knowledge Graph Enhanced Large Language Models

Knowledge graphs (KGs) are widely used to enhance large language models because they explicitly represent entities, relations, and structural dependencies. Existing research has mainly focused on knowledge-enhanced reasoning, retrieval, representation learning, and factuality improvement [23, 24]. Recent surveys further show that KGs and LLM can be combined through KG-augmented LLMs, LLM-augmented KGs, and synergistic frameworks, with evaluation commonly focusing on reasoning, factual consistency, interpretability, and scalability [25, 26].

Early KG research mainly relied on embedding methods to learn entity and relation representations, such as TransE [27] and RotatE [28]. Graph neural networks then extended this line of work by supporting message passing over structured neighborhoods. Among them, relational graph convolutional networks (R-GCNs) [29] are widely used for modeling multi-relational graph structures. More recently, KGEditor [30] has shown that language model-based KG embeddings also require post-deployment editing, indicating that dynamic knowledge updates are important not only for LLM parameters but also for KG representations.

Following this direction, recent studies have begun to incorporate KG structure into LLM knowledge editing. GLAME [31] constructs an edit-related KG subgraph to capture associated knowledge changes caused by a target edit and injects the encoded graph information into a rank-one editing framework. Building on this idea, KGMET [32] further uses an edit-centered subgraph and R-GCN encoding to derive directional information for memory editing while introducing orthogonal constraints and multi-layer updates to improve batch editing, multi-hop reasoning, and the preservation of unrelated knowledge.

In parallel with parameter-modifying KG editing, parameter-preserving approaches use external graph structures to support edited reasoning without directly changing model weights. CAPE-KG [33] separates factual and edited knowledge through a multi-layer KG architecture and uses case-isolated updates with edit-aware retrieval to maintain consistency in multi-hop question answering. HYPE [34] further introduces hyperbolic graph construction and geometry-aware updates to preserve hierarchical relations during lifelong editing. Hybrid-DMKG [35] extends this trend to dynamic multimodal KGs, showing that KG-enhanced editing can also support multimodal multi-hop reasoning and robustness to visually rephrased inputs.

Although these studies demonstrate the value of KGs for knowledge editing, existing methods still face two limitations. First, structural information is often injected through direct representation fusion or score modulation, which may amplify unstable attribution signals. Second, most KG-enhanced editing methods model graph-level associations but do not explicitly constrain whether selected neurons move the hidden state toward the target editing direction. To address these issues, SNKE combines edit-centered KG guidance with target-residual-aware neuron attribution and soft-gated parameter updates, aiming to improve neuron localization, relational consistency, and editing stability.

Existing KG-enhanced editing methods demonstrate that relational structure can improve knowledge consistency, but most of them inject graph information as auxiliary context or post-localization modulation. Less attention has been paid to whether graph structure can directly improve the attribution stage itself, particularly when the editor must distinguish neurons that merely respond to a prompt from neurons whose updates support the target fact.

3. Notation and Preliminaries

This section introduces the notation used in SNKE and reviews the preliminaries of knowledge editing and knowledge graphs.

3.1. Notation

To avoid ambiguity and ensure consistency throughout the text, we have provided descriptions of all mathematical symbols and summarized them in Table 1.

3.2. Preliminaries

3.2.1. Knowledge Editing

First, we define a pre-trained LLM as $f_\theta : \mathcal{X} \to \mathcal{Y}$, where θ denotes the parameters of the pre-trained LLM. $\mathcal{X}$ and $\mathcal{Y}$ are the input and output spaces of the language model. During the knowledge editing process, we transform all factual editing requests $\mathcal{E}$ into structured triples $(s, \rho, o \to o') \in \mathcal{E}$, where s, ρ, o, o' denote the subject, relation, original object, and target object for a single editing request, respectively. The objective of knowledge editing is to update the model parameters from θ to θ' such that the edited model satisfies $f_{\theta'}(s, \rho) = o'$.

An effective knowledge editing method should satisfy the following requirements: (1) **Efficacy**: The edited model should generate the target fact after editing. (2) **Generality**: The edit should remain effective for paraphrased prompts or equivalent queries. (3) **Locality**: The edited model should preserve unrelated knowledge, i.e., $f_\theta(x) = f_{\theta'}(x)$ for $x \notin \mathcal{E}$. Here, $x \in \mathcal{X}$ denotes an input irrelevant to the editing request.

Table 1: Notation and symbol definitions used in the proposed SNKE.

Notation	Description
Editing request and graph notation	
$e = (s, \rho, o, o')$	Editing request with subject s, relation ρ, original object o, and edited object o'.
$\mathcal{E} = \{e_i\}_{i=1}^{N}$	Set of N editing requests.
$\mathcal{G}$	External knowledge graph.
$\mathcal{G}_m^n(e)$	Edit-centered subgraph for request e, with expansion order n and neighbor budget m.
$\mathcal{V}, \mathcal{R}$	Entity set and relation-type set in the knowledge graph.
$\mathcal{N}_s^\rho$	Neighbor set of subject node s under relation ρ.
$c_{s,\rho}$	Normalization factor for RGCN message aggregation.
Hidden representations and structural guidance	
d	LLM hidden dimension.
$\mathbf{h}_{[x]}^k(x)$	Hidden representation of text x at layer k, taken at the last token.
$\mathbf{z}_e \in \mathbb{R}^d$	Original hidden representation at the edit location for request e.
$\boldsymbol{\delta}_e \in \mathbb{R}^d$	Optimized additive editing vector.
$\mathbf{z}'_e \in \mathbb{R}^d$	Target editing representation, $\mathbf{z}'_e = \mathbf{z}_e + \boldsymbol{\delta}_e$.
$\mathbf{r}_e \in \mathbb{R}^d$	Target residual, $\mathbf{r}_e = \mathbf{z}'_e - \mathbf{z}_e$.
$\mathbf{z}_s, \mathbf{z}_\rho, \mathbf{z}_{o'}$	Initial representations of subject, relation, and edited object.
$\mathbf{z}_s^t$	Subject-node representation after the t-th RGCN propagation step.
$\mathbf{W}_\rho^t, \mathbf{W}_0^t$	Relation-specific and self-loop transformation matrices in RGCN.
$\bar{\mathbf{g}}_e \in \mathbb{R}^d$	Unnormalized structural editing direction derived from RGCN.
$\mathbf{g}_e \in \mathbb{R}^d$	Normalized structural editing direction.
α, f_g	Learnable structural scale and graph-guidance scaling hyperparameter.
Neuron scoring and masking	
$l \in \mathcal{L}$	Editable LLM layer; $\mathcal{L}$ is the editable layer set.
$L = \lvert\mathcal{L}\rvert$	Number of editable layers.
d_f	Number of rewrite-module neurons.
$p = 1, \ldots, P$	Prompt index for the current edit.
$a_{p,j}^l$	Activation of neuron j at layer l under prompt p.
$A_{p,j}^l = \lvert a_{p,j}^l \rvert$	Activation strength of neuron j.
$\mathbf{A}_p^l \in \mathbb{R}^{d_f}$	Activation-strength vector for prompt p.
$\mathbf{w}_j^l \in \mathbb{R}^d$	Output direction of neuron j in the rewrite module at layer l.
$\mathbf{W}_{\text{out}}^l \in \mathbb{R}^{d_f \times d}$	Matrix of neuron output directions, with row j equal to $(\mathbf{w}_j^l)^\top$.
$B_{p,j}^l = \lvert\cos(\mathbf{r}_e, \mathbf{w}_j^l)\rvert$	Target-residual alignment score.
$\mathbf{B}_p^l \in \mathbb{R}^{d_f}$	Residual-alignment vector for prompt p.
$\mathbf{Q}_p^l \in \mathbb{R}^{d_f}$	Graph-neuron compatibility vector.
$\widetilde{\mathbf{Q}}_p^l \in \mathbb{R}^{d_f}$	Normalized graph-neuron compatibility vector.
$\mathbf{G}_p^l \in \mathbb{R}^{d_f}$	Graph-guided score gate for prompt p.
$\tilde{\mathbf{s}}_p^l \in \mathbb{R}^{d_f}$	Final graph-guided neuron score vector for prompt p.
$\tilde{\mathbf{S}}^l \in \mathbb{R}^{P \times d_f}$	Prompt-level score matrix stacked from $\{\tilde{\mathbf{s}}_p^l\}_{p=1}^P$.
$\widetilde{\mathbf{S}}^l \in \mathbb{R}^{P \times d_f}$	Row-normalized prompt-level score matrix.
c_j^l	Consistency count of neuron j across prompts.
u_j^l	Peak-response score of neuron j across prompts.
$\tau_{\text{cons}}, \tau_{\text{peak}}$	Adaptive quantile thresholds for consistency and peak-response selection.
$\mathbf{m}_{\text{cons}}^l \in \{0,1\}^{d_f}$	Consistency mask.
$\mathbf{m}_{\text{peak}}^l \in \{0,1\}^{d_f}$	Peak-response mask.
$\mathbf{m}^l \in \{0,1\}^{d_f}$	Final neuron-specific mask.
Null-space constrained update	
$\mathbf{K}^l \in \mathbb{R}^{d_f \times q}$	Key representation matrix at editable layer l.
$\mathbf{Z}', \mathbf{Z}^l \in \mathbb{R}^{d \times q}$	Target and current representation matrices for the update solve.
$\mathbf{R}^l \in \mathbb{R}^{d \times q}$	Layer residual matrix, $\mathbf{R}^l = (\mathbf{Z}' - \mathbf{Z}^l)/(L - l)$.
q	Number of key-value pairs used in the layer-wise update solve.
$\mathbf{P}^l \in \mathbb{R}^{d_f \times d_f}$	Null-space projector at layer l.
$\mathbf{C}^l \in \mathbb{R}^{d_f \times d_f}$	Cached key-statistics matrix.
λ	L2 regularization coefficient.
$\Delta\mathbf{W}^l \in \mathbb{R}^{d_f \times d}$	Solved update matrix in implementation layout.
$\mathbf{H}^l \in \mathbb{R}^{d_f}$	Soft neuron gate applied row-wise to $\Delta\mathbf{W}^l$.
η	Soft-gate strength.
θ, θ'	Original and edited model parameters.

3.2.2. Locate-then-Edit Formalization

Most existing knowledge editing methods follow a typical Locate-then-Edit paradigm, which first identifies model components related to the target knowledge and then performs localized parameter modification. The core assumption behind this paradigm is that factual knowledge in large language models is stored in a sparse subset of neurons or parameters θ inside transformer FFNs. The update formula is expressed as follows.

$$\theta' = \theta + \Delta\theta,$$

where $\Delta\theta$ denotes the localized parameter modification. The goal of knowledge editing is to modify a specific factual association stored in the model while preserving unrelated knowledge and the original generation capability. Therefore, the knowledge editor can achieve the desired knowledge rewriting by optimizing a local update matrix. The optimization objective for the parameter update matrix is defined as:

$$\Delta\mathbf{W}^l = \arg\min_{\hat{\Delta}\mathbf{W}^l} \left(\left\| \left(\mathbf{W}^l + \hat{\Delta}\mathbf{W}^l\right)\mathbf{K}_1 - \mathbf{V}_1 \right\|_2^2 + \left\| \left(\mathbf{W}^l + \hat{\Delta}\mathbf{W}^l\right)\mathbf{K}_0 - \mathbf{V}_0 \right\|_2^2 \right),$$

here $\Delta\theta$ denotes the perturbation to the full model parameters, $\Delta\mathbf{W}_\lambda$ is the final local weight update obtained for the lth layer, $\hat{\Delta}\mathbf{W}^l$ represents the candidate update variable during the optimization process, and W^l is the original weight of the lth layer. $\mathbf{K}_0$ and $\mathbf{V}_0$ are retained key-value pairs. After knowledge editing, these key-value pairs remain unchanged. $\mathbf{K}_1$ and $\mathbf{V}_1$ denote the key-value pairs associated with the edited knowledge. The closed-form solution can be derived:

$$\Delta\mathbf{W}^l = \left(\mathbf{V}_1 - \mathbf{W}^l\mathbf{K}_1\right)\mathbf{K}_1^\top \left(\mathbf{K}_0\mathbf{K}_0^\top + \mathbf{K}_1\mathbf{K}_1^\top\right)^{-1}.$$

3.2.3. Knowledge Graph

A knowledge graph (KG) $\mathcal{G}$ is defined as a set of triples $\{(s, \rho, o) \mid s, o \in \mathcal{V}, \rho \in \mathcal{R}\}$, where $\mathcal{V}$ denotes the entity set and $\mathcal{R}$ denotes the relation set. Editing requests use the same subject-relation-object form. SNKE uses this correspondence to construct edit-centered subgraphs, allowing neuron attribution to access relational evidence around the target fact.

4. Methodology

SNKE is designed to select neurons whose updates are both directionally aligned with the target fact and structurally supported by the edit-centered relational neighborhood. The method first derives a target residual direction, then uses graph-guided attribution to refine neuron scores, and finally applies a soft-gated null-space constrained update.

4.1. Target Editing Representation

Before selecting neurons and updating parameters, SNKE computes the target editing representation $\mathbf{z}'_e$. This vector specifies the hidden-state representation that the edited LLM is expected to produce for the target fact.

Given an editing request $e = (s, r, o, o')$, SNKE constructs rewriting prompts using the subject s and the edited object o'. It then locates the subject token in each prompt and records its original hidden representation at the edit layer as $\mathbf{z}_e$.

The model parameters are frozen during this step. SNKE only optimizes an additive vector $\boldsymbol{\delta}_e$ at the subject token position:

$$\mathbf{h}_s^l \leftarrow \mathbf{h}_s^l + \boldsymbol{\delta}_e.$$

The goal is to make the model predict the new target object o' while keeping the output distribution on unrelated prompts stable. The optimization objective is:

$$\mathcal{L}_z = \mathcal{L}_{\text{NLL}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_\delta\mathcal{L}_\delta.$$

The first term encourages SNKE to generate the edited object:

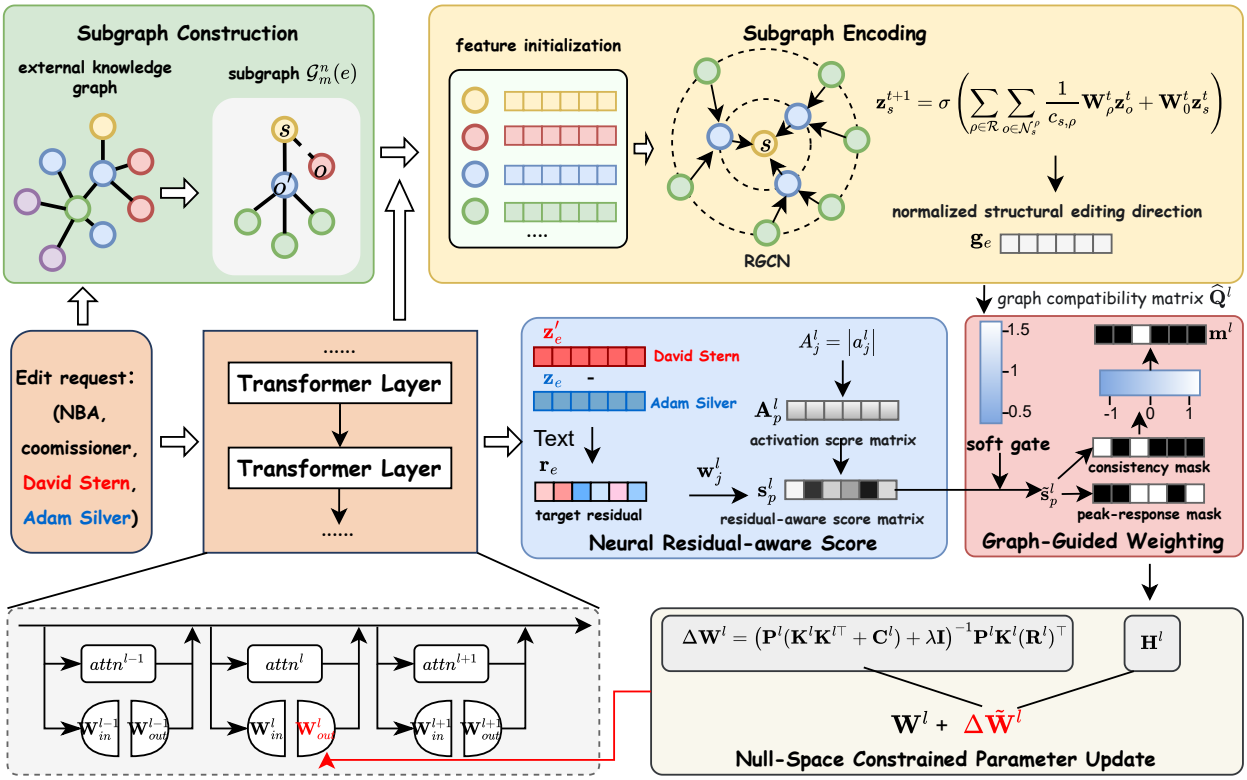


Figure 1: Overall workflow of SNKE. The framework derives a target editing representation, constructs an edit-centered knowledge subgraph, computes graph-guided neuron attribution scores, builds consistency and peak-response masks, and applies a soft-gated null-space constrained parameter update.

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{|T|}\sum_{t\in T}\log p_\theta\left(o'_t \mid x_{<t}; \boldsymbol{\delta}_e\right),$$

where T denotes the target token positions of o'. The second term preserves the original prediction behavior on KL prompts:

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}\left(P_\delta(\cdot \mid x_{\text{KL}}) \,\|\, P_0(\cdot \mid x_{\text{KL}})\right),$$

where P_0 is the original output distribution and P_δ is the output distribution after adding $\boldsymbol{\delta}_e$.

This term penalizes distributional drift on KL prompts and helps preserve unrelated model behavior. The final term regularizes the magnitude of the optimized vector:

$$\mathcal{L}_\delta = \frac{\|\boldsymbol{\delta}_e\|_2^2}{\|\mathbf{z}_e\|_2^2}.$$

After optimization, the target editing representation is obtained by adding the learned residual to the original hidden representation:

$$\mathbf{z}'_e = \mathbf{z}_e + \boldsymbol{\delta}_e.$$

This vector is later used as the target representation for computing the editing residual, training the structural graph direction, and solving the final parameter update.

4.2. Knowledge Graph Augmentation

4.2.1. Subgraph Construction

To guide directional intervention, SNKE constructs an edit-centered knowledge subgraph from an external knowledge graph. The subgraph provides high-order relational context for the edit and helps reduce interference with unrelated facts.

Given an editing request $e = (s, \rho, o, o')$, SNKE aims to replace the original object o with the edited object o'. We therefore treat o' as the semantic anchor of the newly introduced knowledge. Specifically, we first align o' with the most relevant entity in the external knowledge graph. Then, starting from this anchor entity, we sample its neighboring entities and relations. This gives triples such as (o', ρ_1, o_1), (o', ρ_2, o_2), …, (o', ρ_m, o_m). These triples describe the local relational context of the edited object.

Next, we associate this newly introduced object-level knowledge with the original editing subject s. To this end, we compose the neighboring triples with the target edit relation. This forms edit-induced high-order associations such as $(s, \rho, o', \rho_1, o_1)$, $(s, \rho, o', \rho_2, o_2)$, …, $(s, \rho, o', \rho_m, o_m)$. The resulting subgraph contains the edited fact (s, ρ, o') and captures relational knowledge that becomes relevant after the edit. By recursively sampling the neighbors of o_1, o_2, ..., o_m, we can further expand the graph. This expansion includes higher-order associations centered on the edited knowledge.

We denote the resulting subgraph as $\mathcal{G}_m^n(e)$. Here, n is the maximum expansion order, and m is the maximum number of

sampled neighbors for each entity. These two hyperparameters jointly control the scale and density of the constructed subgraph. The subgraph $\mathcal{G}_m^n(e)$ consists of the target edited fact and its induced high-order relational context. It provides structured guidance for modeling the knowledge changes caused by the edit.

4.2.2. Subgraph Encoding

The constructed subgraph $\mathcal{G}_m^n(e)$ contains the edited fact and its high-order relational context. SNKE encodes this structured context into graph representations, which are later used to guide neuron selection and directional intervention. This allows SNKE to incorporate edit-relevant high-order knowledge while suppressing interference from unrelated facts.

We first initialize the node and relation features with the hidden states of the LLM. Specifically, we use the hidden-state representations from the k-th layer as the initial features:

$$\mathbf{z}_s = \mathbf{h}_{[s]}^k(s), \quad \mathbf{z}_\rho = \mathbf{h}_{[\rho]}^k(\rho), \quad \mathbf{z}_{o'} = \mathbf{h}_{[o']}^k(o'),$$

where $\mathbf{h}_{[x]}^k(x)$ denotes the hidden state (the last token) of text x at the k-th layer. For the neighboring entities and relations in $\mathcal{G}_m^n(e)$, their initial features are obtained in the same way.

Next, SNKE applies a Relational Graph Convolutional Network (RGCN) to aggregate information from the edit-centered subgraph. For the subject node s, the propagation step is:

$$\mathbf{z}_s^{t+1} = \sigma\left(\sum_{\rho\in\mathcal{R}}\sum_{o\in\mathcal{N}_s^\rho}\frac{1}{c_{s,\rho}}\mathbf{W}_\rho^t\mathbf{z}_o^t + \mathbf{W}_0^t\mathbf{z}_s^t\right).$$

Here, $\mathcal{N}_s^\rho$ denotes the set of neighboring object nodes connected to subject s under relation ρ, and $c_{s,\rho}$ is a normalization factor. $\mathbf{z}_s^t$ and $\mathbf{z}_o^t$ denote the representations of the subject node and its neighboring object node at layer t, respectively. $\mathbf{W}_\rho^t$ is the relation-specific transformation matrix, $\mathbf{W}_0^t$ denotes the self-loop transformation matrix, and $\sigma(\cdot)$ is the activation function.

After n layers of relational message propagation, SNKE obtains the updated subject representation $\mathbf{z}_s^n$. It first normalizes this representation and scales it with a learnable factor:

$$\bar{\mathbf{g}}_e = \alpha \cdot f_g \cdot \mathrm{Norm}(\mathbf{z}_s^n),$$

where α is a learnable scalar and f_g is a graph-guidance scaling hyperparameter.

To align the graph-derived direction with the target editing representation, SNKE optimizes:

$$\mathcal{L}_{\mathrm{RGCN}} = \mathrm{MSE}\left(\bar{\mathbf{g}}_e, \mathbf{z}_e'\right) = \frac{1}{d}\left\|\bar{\mathbf{g}}_e - \mathbf{z}_e'\right\|_2^2,$$

where $\mathbf{z}_e'$ denotes the target editing representation obtained from the target-representation optimization process. After optimization, the structural guidance direction is obtained as:

$$\mathbf{g}_e = \mathrm{Norm}(\bar{\mathbf{g}}_e).$$

4.3. Neuron-specific Mask

4.3.1. Neural Residual-aware Score

After obtaining the target editing representation $\mathbf{z}_e'$, SNKE identifies the neurons most relevant to the current edit. Instead of updating all neurons uniformly, SNKE builds a neuron-specific mask for each editable layer and later uses this mask to softly gate the parameter update.

For an editing request e, let $\mathbf{z}_e$ be the current hidden representation of the subject at the target layer. The target residual is defined as:

$$\mathbf{r}_e = \mathbf{z}_e' - \mathbf{z}_e.$$

Here, $\mathbf{r}_e$ describes the hidden-state direction required by the edit.

For each editable layer l, SNKE collects the input activation of the rewrite module. For each prompt p, the initial neuron score is defined as:

$$A_j^l = \left|a_j^l\right|,$$

where a_j^l is the activation of neuron j at layer l on prompt p, and the corresponding neuron score for prompt p is defined as $\mathbf{A}_p^l = [A_{p,1}^l, ..., A_{p,d_f}^l]$; d_f is the maximum number of rewrite-module neurons.

Then, SNKE aligns this activation score with the target residual. Let $\mathbf{w}_j^l$ denote the output vector of neuron j in the rewrite matrix of layer l. The residual alignment score for each neuron B_j^l is defined as:

$$B_j^l = \left|\cos\left(\mathbf{r}_e, \mathbf{w}_j^l\right)\right|.$$

The corresponding residual alignment score vector for all neurons is defined as $\mathbf{B}^l = [B_1^l, B_2^l, ..., B_{d_f}^l]$.

4.3.2. Graph-Guided Weighting

The residual-aware scores above reflect LLM activations and their alignment with the target residual. To further incorporate high-order relational knowledge, SNKE uses the structural editing direction $\mathbf{g}_e$ obtained from the RGCN-encoded edit-centered subgraph. This direction reweights neuron importance scores before the final mask is constructed.

For each editable layer l, let $\mathbf{W}_{\mathrm{out}}^l = [\mathbf{w}_1^l; \cdots; \mathbf{w}_{d_f}^l] \in \mathbb{R}^{d_f \times d}$ denote the matrix of neuron output directions. SNKE computes the graph-neuron compatibility vector as

$$\mathbf{Q}^l = \sigma\left(\mathbf{W}_{\mathrm{out}}^l\mathbf{g}_e\right),$$

where $\sigma(\cdot)$ denotes the sigmoid function. The compatibility scores $\hat{\mathbf{Q}}^l$ are then normalized within each prompt:

$$\hat{\mathbf{Q}}^l = \frac{\mathbf{Q}^l - \mathrm{mean}(\mathbf{Q}^l)}{\mathrm{std}(\mathbf{Q}^l) + \epsilon},$$

SNKE then converts the normalized graph score into a soft gate:

$$\mathbf{G}^l = \min\left(\max\left(\mathbf{1} + \gamma\hat{\mathbf{Q}}^l, 0.5\right), 1.5\right),$$

where γ is the graph-guidance strength. The clamp operation restricts the graph-guided gate to a moderate range. This prevents the structural guidance from overwhelming the activation- and residual-based importance scores. In practice, the gate can scale the score to between 0.5 and 1.5 times its original value, which allows the graph signal to softly adjust neuron importance while maintaining stable neuron selection.

For a single prompt p, the graph-guided neuron score vector is defined as follows:

$$\tilde{\mathbf{s}}_p^l = \mathbf{A}_p^l \odot \mathbf{B}_p^l \odot \mathbf{G}_p^l.$$

This vector integrates three types of evidence: activation strength $\mathbf{A}_p^l$, target-residual alignment $\mathbf{B}_p^l$, and graph-guided structural consistency $\mathbf{G}_p^l$. When multiple prompts are used for the same edit, SNKE stacks their score vectors into $\tilde{\mathbf{S}}^l = [\tilde{\mathbf{s}}_1^l; \tilde{\mathbf{s}}_2^l; \cdots; \tilde{\mathbf{s}}_P^l] \in \mathbb{R}^{P \times d_f}$.

Because edit-related neurons may either remain consistently important across prompts or respond sharply to a specific prompt, SNKE constructs two complementary masks from the graph-guided scores: the consistency mask $\mathbf{m}_{\text{cons}}^l$ and the peak-response mask $\mathbf{m}_{\text{peak}}^l$. Here, consistency neurons denote neurons whose attribution scores remain stably aligned with the target editing direction across prompts (This term is distinct from the generation consistency metric Consis. used in the experiments.) Peak-response neurons denote neurons that exhibit strong response peaks under the target editing signal. The corresponding consistency mask and peak-response mask are used to guide soft-gated parameter updates.

For the consistency mask, SNKE first counts how often each neuron receives a positive normalized score across prompts:

$$c_j^l = \sum_p \mathbb{I}\left[\hat{S}_{p,j}^l > 0\right],$$

where $\hat{S}_{p,j}^l$ denotes the row-normalized graph-guided score for neuron j under prompt p. The consistency mask is then defined as

$$(\mathbf{m}_{\text{cons}}^l)_j = \mathbb{I}\left[c_j^l \geq \tau_{\text{cons}}\right],$$

where τ_{cons} is an adaptive quantile threshold over the neuron-wise consistency counts. In parallel, the peak-response mask selects neurons with strong prompt-specific responses:

$$u_j^l = \max_p \tilde{S}_{p,j}^l, \qquad (\mathbf{m}_{\text{peak}}^l)_j = \mathbb{I}\left[u_j^l \geq \tau_{\text{peak}}\right],$$

where τ_{peak} is an adaptive quantile threshold over the neuron-wise peak-response scores. The thresholds τ_{cons} and τ_{peak} are not fixed manually; they are selected according to the score distribution of the current editable layer.

Therefore, the final mask retains neurons that are consistently important across prompts, as well as neurons with strong prompt-specific responses. Because both masks are derived from $\tilde{\mathbf{S}}^l$, the selected neurons remain important for the target residual and aligned with the high-order relational direction provided by the subgraph. The formula for the final neuron-specific mask $\mathbf{m}^l$ is shown as follows:

$$\mathbf{m}^l = \min\left(\mathbf{m}_{\text{cons}}^l + \mathbf{m}_{\text{peak}}^l, 1\right).$$

4.4. Null-Space Constrained Parameter Update

The neuron mask controls where the update is emphasized, whereas the null-space constraint controls which parameter directions are allowed. SNKE therefore combines these two mechanisms to localize the update both at the neuron level and in the parameter subspace. It then computes the parameter update for each editable layer. The goal is to write the target residual into the model while reducing interference with unrelated knowledge.

Let $\mathbf{K}^l$ denote the key representations produced by the editing prompts at layer l. Let $\mathbf{R}^l$ denote the residuals that should be written into the same layer. In the implementation, the residual is divided among the remaining editable layers:

$$\mathbf{R}^l = \frac{\mathbf{Z}' - \mathbf{Z}^l}{L - l},$$

where $\mathbf{Z}' = [\mathbf{z}'_{e_1} \mid \mathbf{z}'_{e_2} \mid \cdots \mid \mathbf{z}'_{e_N}]$ denotes the stack for the target editing representations, $\mathbf{Z}^l = [\mathbf{z}^l_{e_1} \mid \mathbf{z}^l_{e_2} \mid \cdots \mid \mathbf{z}^l_{e_N}]$ denotes the stack of current representations at layer l, L is the number of editable layers, and l is the current layer.

To reduce interference with unrelated knowledge, SNKE introduces a null-space projector $\mathbf{P}^l$. This projector is computed from the covariance statistics of the rewrite module. Singular directions whose singular values are below a threshold are kept to form the null-space projector.

The constrained update is computed as:

$$\Delta\mathbf{W}^l = \left(\mathbf{P}^l(\mathbf{K}^l\mathbf{K}^{l\top} + \mathbf{C}^l) + \lambda\mathbf{I}\right)^{-1} \mathbf{P}^l\mathbf{K}^l(\mathbf{R}^l)^\top,$$

where $\mathbf{C}^l$ is a cache matrix that stores previous key statistics, λ is the L2 regularization coefficient, and $\mathbf{I}$ is the identity matrix. In this equation, $\Delta\mathbf{W}^l$ denotes the update matrix in the linear-solve layout, where each row corresponds to one rewrite-module neuron. This layout is used for computing and gating the update. After the solve, the update is transposed or reshaped when necessary to match the actual layout of the rewrite weight.

Finally, the graph-guided neuron mask softly gates the update. The gate is:

$$\mathbf{H}^l = \mathbf{1} + \eta\left(\mathbf{m}^l - \text{mean}(\mathbf{m}^l)\right),$$

where η is the soft-gate strength. Since $\mathbf{m}^l \in \{0,1\}^{d_f}$, the gate $\mathbf{H}^l \in \mathbb{R}^{d_f}$ assigns one scaling coefficient to each rewrite-module neuron. The gated update is:

$$\Delta\tilde{\mathbf{W}}^l = \text{diag}(\mathbf{H}^l)\Delta\mathbf{W}^l.$$

Here, $\text{diag}(\mathbf{H}^l)$ converts the neuron-wise gate into a diagonal scaling matrix so that the j-th row of $\Delta\mathbf{W}^l$ is scaled by H_j^l. Before being added to the model parameters, $\Delta\tilde{\mathbf{W}}^l$ is transposed or reshaped when necessary to match the original parameter layout. The final parameter update is:

$$\mathbf{W}_{\text{out}}^l \leftarrow \mathbf{W}_{\text{out}}^l + \Delta\tilde{\mathbf{W}}^l.$$

The procedure scores neurons with activation and target-residual alignment, refines the scores using graph-based structural guidance, and applies a null-space constrained update through a soft neuron gate.

The overall workflow of SNKE is shown in Algorithm 1:

Algorithm 1: Algorithmic procedure of SNKE for graph-aware neuron-level knowledge editing.

Input: Pre-trained model f_θ; editing requests $\mathcal{E} = \{e_i = (s_i, \rho_i, o_i, o_i')\}_{i=1}^{N}$; external knowledge graph $\mathcal{G}$; editable layer set $\mathcal{L}$

Output: Edited model $f_{\theta'}$

foreach *editing request* $e_i = (s_i, \rho_i, o_i, o_i') \in \mathcal{E}$ **do**
- Construct rewriting prompts using s_i and o_i';
- Optimize the additive vector $\boldsymbol{\delta}_{e_i}$ with frozen model parameters;
- Obtain the target editing representation $\mathbf{z}'_{e_i} \leftarrow \mathbf{z}_{e_i} + \boldsymbol{\delta}_{e_i}$;
- Construct the edit-centered subgraph $\mathcal{G}_m^n(e_i)$ from $\mathcal{G}$;
- Encode $\mathcal{G}_m^n(e_i)$ with RGCN and obtain the structural direction $\mathbf{g}_{e_i}$;

Stack target representations: $\mathbf{Z}' \leftarrow [\mathbf{z}'_{e_1} \mid \mathbf{z}'_{e_2} \mid \cdots \mid \mathbf{z}'_{e_N}]$;

foreach *editable layer* $l \in \mathcal{L}$ **do**
- Extract key matrix $\mathbf{K}^l$ and current representations $\mathbf{Z}^l$;
- Compute the layer residual $\mathbf{R}^l \leftarrow \frac{\mathbf{Z}' - \mathbf{Z}^l}{L-l}$;
- **foreach** *prompt p associated with the current edit* **do**
  - Compute activation-strength vector $\mathbf{A}_p^l$;
  - Compute target-residual alignment vector $\mathbf{B}_p^l$;
  - Compute graph-neuron compatibility $\mathbf{Q}_p^l \leftarrow \sigma(\mathbf{W}_{\text{out}}^l \mathbf{g}_{e_i})$;
  - Normalize $\mathbf{Q}_p^l$ and compute graph-guided gate $\mathbf{G}_p^l$;
  - Compute graph-guided neuron score $\tilde{\mathbf{s}}_p^l \leftarrow \mathbf{A}_p^l \odot \mathbf{B}_p^l \odot \mathbf{G}_p^l$;
- Merge masks $\mathbf{m}^l \leftarrow \min(\mathbf{m}_{\text{cons}}^l + \mathbf{m}_{\text{peak}}^l, 1)$;
- Solve the null-space constrained update $\Delta\mathbf{W}^l \leftarrow \left(\mathbf{P}^l(\mathbf{K}^l\mathbf{K}^{l\top} + \mathbf{C}^l) + \lambda\mathbf{I}\right)^{-1} \mathbf{P}^l\mathbf{K}^l(\mathbf{R}^l)^\top$;
- Compute soft neuron gate $\mathbf{H}^l \leftarrow \mathbf{1} + \eta(\mathbf{m}^l - \text{mean}(\mathbf{m}^l))$;
- Apply row-wise gated update $\Delta\tilde{\mathbf{W}}^l = \text{diag}(\mathbf{H}^l)\Delta\mathbf{W}^l$;
- Update rewrite weight after matching the parameter layout $\mathbf{W}_{\text{out}}^l \leftarrow \mathbf{W}_{\text{out}}^l + \Delta\tilde{\mathbf{W}}^l$;

return $f_{\theta'}$;

5. Experiments

5.1. Datasets

We evaluate SNKE on three benchmarks with different stress points. ZsRE [36] tests question-answer style factual edits and paraphrase-level generality. MultiCounterFact [10] stresses counterfactual editing on a larger scale, where edited facts are more likely to interact with neighboring associations. MQuAKE [37] moves beyond direct rewrite accuracy and asks whether an edit propagates through two-hop and three-hop reasoning chains.

ZsRE [36] is a question-answering benchmark derived from zero-shot relation extraction. Each example can be viewed as an editing request that asks the model to associate a subject with a target answer under a relation-specific question. In our experiments, ZsRE is mainly used to evaluate reliability and generality. Reliability measures whether the edited model produces the new target answer for the editing prompt, while generality measures whether the edit transfers to semantically related question forms. Since ZsRE examples are expressed as natural-language questions, this dataset is useful for evaluating whether SNKE can perform fact editing in a QA-style setting rather than only in cloze-style factual statements.

MultiCounterFact [10] is used to evaluate SNKE under a larger and more challenging factual editing setting. Compared with the standard CounterFact [9] setting, MultiCounterFact contains more editing records and is suitable for testing batch editing and robustness under multiple factual updates. In our experiments, it is used to examine whether SNKE can maintain editing performance when many counterfactual associations are edited. This setting is closely related to the motivation of our method because graph-guided neuron selection and null-space constrained updates are designed to reduce interference when multiple edits are injected into the same model.

MQuAKE [37] is a multi-hop question-answering benchmark for knowledge editing. Unlike datasets that evaluate only the directly edited fact, MQuAKE asks whether the model can answer questions whose correct answers change as a consequence of one or more edited facts. We use MQuAKE to evaluate higher-order relational consistency and portability. This dataset is relevant to SNKE because SNKE constructs an edit-centered knowledge subgraph and encodes higher-order relational context with RGCN. Therefore, MQuAKE provides a direct test of whether the structural guidance introduced by SNKE helps the model update not only the target fact but also the related multi-hop knowledge induced by the edit.

The statistics of the datasets used in our local evaluation files are summarized in Table 2. The numbers correspond to the records loaded by the implementation in the current experimental setting.

These datasets cover direct factual updates, large-scale counterfactual associations, and edit-induced multi-hop dependencies. In Table 2, MQuAKE contains 3,000 records, while the multi-hop experiment uses the first 2,000 cases for evaluation.

5.2. Experimental Setup

5.2.1. Baseline Models

The baselines cover the main editing paradigms relevant to SNKE. They include direct fine-tuning, meta-learning-based editing, locate-and-edit updates, preservation-oriented editing, graph-enhanced editing, and neuron-level sparse editing. This design compares SNKE with methods that modify similar model components and with methods based on different assumptions about where edited knowledge is stored. We evaluate the methods on LLaMA 3 [38], GPT-J-6B, and GPT-2-XL

Table 2: Dataset statistics and evaluation focus used in SNKE experiments. The table reports each benchmark's task type, primary evaluation focus, and number of loaded records.

Dataset	Task Type	Evaluation Focus	Number of Edits
ZsRE	Question-answer editing	Reliability and Generality	19,086
MultiCounterFact	Large-scale factual association editing	Batch editing, robustness, and interference control	20,877
MQuAKE	Multi-hop question-answer editing	High-order relational consistency and portability	3,000

to examine whether the observed editing behavior is consistent across different model scales and architectures [39, 15].

Pre-edited: The pre-edited model refers to the original language model before any parameter update is applied. It provides the reference behavior before editing. This baseline is used to evaluate whether an editor changes the target knowledge while preserving unrelated facts and general language performance.

FT: Fine-tuning [40] directly updates the model parameters using the editing examples. In this setting, the model is optimized to increase the likelihood of the new target answer. FT provides a simple and intuitive baseline for knowledge editing. However, because it does not explicitly localize the update or constrain the changed parameter directions, it may overfit to the editing examples and cause interference with unrelated knowledge.

MEND: MEND [8] edits model behavior through a learned auxiliary network that transforms gradients into parameter updates. Instead of directly applying standard gradient descent to the original model, MEND learns how to produce efficient update directions from the editing signal. It is included as a representative meta-learning-based editing method. MEND is useful for evaluating whether learned gradient transformations can provide better edit efficiency and generality than direct fine-tuning.

ROME: ROME [9] is a locate-and-edit method that treats factual knowledge as key-value associations stored in transformer feed-forward layers. It first identifies a critical layer and then performs a rank-one update to modify the value associated with the target subject-relation pair. ROME is mainly designed for single-fact editing. It provides a strong baseline for testing whether SNKE can match or improve direct factual editing while further improving relational generality and locality.

MEMIT: MEMIT [10] extends the locate-and-edit paradigm to batch editing. It distributes updates across multiple layers and provides a closed-form update for many factual associations. MEMIT is an important baseline because SNKE also performs multi-layer parameter updates. The comparison with MEMIT helps isolate the effect of the additional graph-guided neuron selection and null-space constraint introduced by SNKE.

PRUNE: PRUNE [41] is included as a preservation-oriented editing baseline. Its goal is to reduce unnecessary changes to the original model while still achieving the target edit. Compared with methods that mainly optimize editing success, PRUNE emphasizes controlling the influence of editing on unrelated knowledge. This makes it relevant for evaluating the locality and specificity of SNKE.

RECT: RECT [42] is used as another baseline designed to improve the reliability and preservation of model editing. It attempts to reduce undesired side effects during parameter modification and improve the quality of the edited model's behavior. We include RECT to compare SNKE with methods that explicitly consider edit stability and interference reduction.

GLAME: GLAME [43] is a knowledge-enhanced editing baseline that incorporates structured or graph-related information into the editing process. It is especially relevant to SNKE because both methods consider external relational knowledge. However, SNKE differs from GLAME by using an edit-centered subgraph to derive a structural editing direction and by applying this direction to neuron-level scoring and masked parameter updates.

KGMET: KGMET [32] is the most direct knowledge-graph-based baseline for SNKE. It uses knowledge graph information to guide directional intervention and introduces constraints to reduce the influence of irrelevant knowledge. We include KGMET to evaluate whether SNKE improves upon the previous graph-guided editing framework. Compared with KGMET, SNKE further introduces target-residual-aware neuron attribution, graph-guided neuron reweighting, neuron-specific masking, and null-space constrained soft-gated updates.

NMKE: NMKE [13] is a neuron-specific masked knowledge editing method. It selects edit-relevant neurons and restricts updates to a sparse subset of MLP neurons. NMKE is highly relevant to SNKE because both methods aim to reduce interference through neuron-level selection. The key difference is that SNKE combines neuron-level attribution with edit-centered knowledge graph guidance. Thus, the comparison with NMKE helps evaluate whether structural relational information provides additional benefits beyond neuron-specific sparse editing.

5.2.2. Evaluation Protocol

We evaluate SNKE from several complementary perspectives: edit efficacy, generality, locality preservation, generation fluency, generation consistency, and high-order relational portability. Efficacy measures whether the target fact is updated, generality measures transfer to equivalent prompts, and locality measures the preservation of unrelated knowledge. For MultiCounterFact, Flu. and Consis. further evaluate the diversity and semantic consistency of generated text, while MQuAKE evaluates portability over multi-hop relational chains [39]. These metrics are aligned with the implementation used for ZsRE and MultiCounterFact.

Let $f_{\theta'}$ denote the edited model. For an editing request $e_i = (s_i, \rho_i, o_i, o_i')$, o_i' is the edited target object, and o_i is the

original object. Given a prompt x, we denote the model probability of an answer y as $P_{\theta'}(y \mid x)$. In the implementation, token-level negative log-likelihood is computed for both o_i' and o_i, and a lower negative log-likelihood indicates a higher model preference.

Efficacy: Efficacy (Eff.) measures whether the edited model successfully recalls the new target object under the original editing prompt. Let $\mathcal{P}_{\text{edit}}$ be the set of rewrite prompts. For MultiCounterFact, an edit is counted as successful when the edited target receives a higher probability than the original object:

$$\text{Eff} = \frac{1}{|\mathcal{P}_{\text{edit}}|} \sum_{x \in \mathcal{P}_{\text{edit}}} \mathbb{I}\left[P_{\theta'}(o_i' \mid x) > P_{\theta'}(o_i \mid x)\right]$$

Generality: Generality (Gen.) measures whether the edited knowledge transfers to prompts that are semantically equivalent to the original editing prompt. Let $\mathcal{P}_{\text{para}}$ denote the paraphrase prompt set. The generality score is:

$$\text{Gen} = \frac{1}{|\mathcal{P}_{\text{para}}|} \sum_{x \in \mathcal{P}_{\text{para}}} \mathbb{I}\left[P_{\theta'}(o_i' \mid x) > P_{\theta'}(o_i \mid x)\right]$$

Locality: Locality (Loc.) measures whether the edit preserves unrelated or neighboring facts. Let $\mathcal{P}_{\text{loc}}$ be the neighborhood prompt set. In these prompts, the original answer should remain preferred. Therefore, locality is computed as:

$$\text{Loc} = \frac{1}{|\mathcal{P}_{\text{loc}}|} \sum_{x \in \mathcal{P}_{\text{loc}}} \mathbb{I}\left[P_{\theta'}(o_i \mid x) > P_{\theta'}(o_i' \mid x)\right]$$

Fluency: Fluency (Flu.) evaluates whether the edited model can still generate fluent and diverse text after knowledge editing. Using the same generation protocol as the generality evaluation, we compute Flu. with the weighted n-gram entropy of generated continuations. The n-gram entropy is computed as:

$$H_n(g_j) = -\sum_{u \in \mathcal{V}_n(g_j)} p_n(u \mid g_j) \log_2 p_n(u \mid g_j).$$

where $\mathcal{G} = \{g_j\}_{j=1}^{M}$ denotes the generated texts produced by the edited model under the generation prompts. $\mathcal{V}_n(g_j)$ represents the set of distinct n-grams in g_j, and $p_n(u \mid g_j)$ is the empirical frequency of an n-gram u normalized over all n-grams in g_j.

In the implementation, Flu. combines the 2-gram and 3-gram entropies with weights $w_2 = \frac{2}{3}$ and $w_3 = \frac{4}{3}$, and then averages over all generated texts:

$$\text{Flu.} = \frac{1}{M} \sum_{j=1}^{M} \frac{1}{2} \left(w_2 H_2(g_j) + w_3 H_3(g_j)\right).$$

A higher Flu. score indicates that the edited model produces less repetitive and more linguistically diverse generations.

Consistency: Consistency (Consis.) measures whether the generated text remains semantically consistent with reference texts related to the edited target knowledge. Consis. is computed as their cosine similarity:

$$\text{Consis.} = \frac{\phi(G)^\top \phi(R_c)}{\|\phi(G)\|_2 \, \|\phi(R_c)\|_2},$$

where R denotes the set of reference texts associated with the target relation and edited object. $\phi(\cdot)$ denotes the TF-IDF vectorizer. A higher Consis. score means that the edited model's generations are more consistent with the reference evidence associated with the edited knowledge.

5.2.3. Implementation Details

SNKE is implemented in PyTorch and evaluated on a single NVIDIA A800 GPU. For a controlled comparison with KGMET, we use the same editing-layer range and graph-parameter settings:

We set editing layers [13, 14, 15, 16, 17, 18] for GPT-2-XL, layers [3 ,4, 5, 6, 7, 8] for GPT-J-6B, and layers [4, 5, 6, 7, 8] for LLaMA 3. We also sample different subgraph sizes in different LLMs. For LLaMA 3 and GPT-J-6B, we sample 40 neighbors for each subgraph and 20 neighbors for GPT-2-XL.

We use the same knowledge graph source as KGMET, obtained from `https://github.com/FredJDean/KGMET`. Since the released repository does not include a ZsRE-specific knowledge graph, we reconstruct the ZsRE graph using the baseline construction procedure and re-run KGMET and NMKE in the same experimental environment. The remaining main-experiment results are taken from the original KGMET paper.

5.3. Main Results and Analysis

The main results compare SNKE with representative editing baselines on LLaMA 3, GPT-J-6B, and GPT-2-XL. MultiCounterFact evaluates counterfactual editing and generation behavior, while ZsRE evaluates question-answer style factual updates.

The main results show that SNKE is most consistent on edit efficacy and generality, especially when edited knowledge must remain valid beyond the exact rewrite prompt. Across the tested backbones, SNKE achieves the best or near-best Eff. and Gen. scores on both MultiCounterFact and ZsRE. This pattern is consistent with the target-residual attribution design: selected neurons are favored when their update directions support the edited fact across alternative prompts and answer forms.

The advantage is clearer on ZsRE because each target answer is usually tied to a bounded question-answer form. In this setting, the target residual direction gives a cleaner signal for strengthening answer-related neurons. MultiCounterFact is less controlled: counterfactual edits can overlap with neighboring entity associations and open-ended generation behavior. This explains why SNKE improves Gen. and Consis. but does not always obtain the highest Loc.

The Flu. results show a related trade-off. Structural guidance keeps generations closer to the edited relation, improving Consis.; however, tighter factual alignment can reduce weighted n-gram entropy. SNKE therefore favors factual consistency and prompt robustness, while KGMET or the pre-edited model may retain higher surface-level fluency in some settings.

5.4. Multi-hop Evaluation

We use MQuAKE to examine whether edited knowledge supports compositional reasoning over relational chains. The

Table 3: Main editing results on MultiCounterFact and ZsRE across LLaMA 3, GPT-J-6B, and GPT-2-XL. Higher scores indicate better performance for all reported metrics. **Bold font** and underlining are employed to denote the best and second-best results, respectively.

Model	Method	MultiCounterFact					ZsRE		
		Eff.↑	Gen.↑	Loc.↑	Flu.↑	Consis.↑	Eff.↑	Gen.↑	Loc.↑
LLaMA 3	Pre-edited	7.85	10.58	89.48	635.23	24.14	36.99	36.34	31.89
	FT	83.33	67.79	46.63	233.72	8.77	30.48	30.22	15.49
	MEND	63.24	61.17	45.37	372.16	4.21	0.91	1.09	0.53
	ROME	64.40	61.42	49.44	449.06	3.31	2.01	1.80	0.69
	MEMIT	65.65	64.65	51.56	437.43	6.58	34.62	31.28	18.49
	PRUNE	68.25	64.75	49.82	418.03	5.90	24.77	23.87	20.69
	RECT	66.05	63.62	61.41	526.62	20.54	86.05	80.54	31.67
	KGMET	**99.20**	92.12	**76.49**	**619.05**	28.86	95.20	**89.71**	31.76
	NMKE	97.15	86.68	71.39	-	-	91.87	87.58	33.02
	SNKE	99.15	**94.25**	67.95	602.06	**33.48**	**96.24**	89.56	**33.11**
GPT-J-6B	Pre-edited	16.22	18.56	83.11	621.81	29.74	26.32	25.79	27.42
	FT	92.15	72.38	43.35	297.92	6.65	72.37	68.91	19.66
	MEND	46.15	46.22	53.90	242.41	3.94	0.71	0.71	0.52
	ROME	57.50	54.20	52.05	589.42	3.22	56.42	54.65	9.86
	MEMIT	98.55	95.50	63.64	546.28	34.89	94.91	90.22	30.39
	PRUNE	86.15	86.85	53.87	427.14	14.78	0.15	0.15	0.00
	RECT	98.80	86.58	72.22	617.31	**41.39**	96.38	91.21	27.79
	KGMET	**99.75**	90.58	**81.48**	**622.33**	38.20	97.95	93.25	28.22
	NMKE	99.65	93.22	74.98	-	-	98.37	92.09	**29.43**
	SNKE	**99.75**	**95.98**	75.80	573.47	26.07	**99.57**	**96.41**	28.27
GPT-2-XL	Pre-edited	22.23	24.34	78.53	626.64	31.88	22.19	31.30	24.15
	FT	63.55	42.20	57.06	519.35	10.56	37.11	33.30	10.36
	MEND	50.80	50.80	49.20	407.21	1.01	0.00	0.00	0.00
	ROME	54.60	51.18	52.68	366.13	0.72	47.50	43.56	14.27
	MEMIT	94.70	85.82	60.50	477.26	22.72	79.17	71.44	26.42
	PRUNE	82.05	78.55	53.02	530.47	15.93	21.62	19.27	13.19
	RECT	92.15	81.15	65.13	480.83	21.05	81.02	73.08	24.85
	KGMET	99.15	83.55	**74.93**	**621.74**	38.52	90.75	79.58	25.45
	NMKE	97.00	86.70	67.81	-	-	84.93	76.91	**26.38**
	SNKE	**99.55**	**92.72**	65.61	518.65	**40.35**	**95.57**	**86.85**	26.13

evaluation is conducted on GPT-2-XL, GPT-J-6B, and LLaMA 3, using 2,000 MQuAKE cases with an editing step size of 50. After each editing group is applied, the edited model is evaluated on MQuAKE portability questions in both 2-hop and 3-hop settings. The evaluation results are shown in Table 4.

SNKE achieves the highest average score across all three backbones and shows consistent improvement in the more difficult 3-hop setting. The gain varies across backbones, but the trend supports the role of graph-aware neuron attribution. By incorporating local relational structure before neuron selection, SNKE is less likely to treat the target fact as an isolated prompt-answer pair. The improvement over ROME and MEMIT suggests that direct parameter updates alone are insufficient for stable reasoning-chain propagation, while the improvement over graph-enhanced baselines indicates that structural information is more useful when it is coupled with target-direction attribution.

Table 4: Multi-hop results on MQuAKE using GPT-2-XL, GPT-J-6B, and LLaMA 3. The table reports the average score and the separate 2-hop and 3-hop reasoning scores under 2,000 edits. **Bold font** denotes the best results.

LLM	Method	Average Score	2-hops	3-hops
GPT-2 XL	Pre-trained	24.22	25.13	23.3
	ROME	35.44	39.8	31.07
	MEMIT	31.79	35.87	27.7
	AlphaEdit	34.37	36.48	32.26
	GLAME	36.97	41.83	32.1
	KGMET	38.32	38.37	38.27
	SNKE	**42.32**	**45.43**	**39.20**
GPT-J	Pre-trained	19.70	15.80	23.60
	ROME	40.59	42.80	38.37
	MEMIT	34.40	35.77	33.03
	AlphaEdit	32.85	34.32	31.37
	GLAME	42.00	44.13	39.87
	KGMET	47.02	48.13	45.90
	SNKE	**48.87**	**51.73**	**46.00**
LLaMA 3	Pre-trained	22.06	22.33	21.78
	ROME	37.98	38.13	37.82
	MEMIT	32.26	32.35	32.17
	AlphaEdit	32.49	33.43	31.54
	GLAME	42.16	40.33	43.99
	KGMET	44.49	44.2	44.77
	SNKE	**46.63**	**44.23**	**49.03**

5.5. Computational Cost and Scalability Analysis

The scalability analysis examines whether the structural and attribution components in SNKE introduce substantial overhead. We compare KGMET, NMKE, and SNKE on GPT-2-XL with 2,000 MultiCounterFact edits and an editing step size of 100. We record total runtime, chunk time, edit time, and peak GPU memory during the editing run, including model parameters and intermediate states used for attribution and updating. The results are shown in Table 5.

SNKE reduces both total runtime and average edit time compared with KGMET, indicating that graph-aware neuron selection does not become the runtime bottleneck in this setting. NMKE remains the fastest method because it uses a simpler masking pipeline, but its lower main-experiment performance shows that runtime should be interpreted together with edit efficacy and generality. The main cost of SNKE is memory. Both NMKE and SNKE maintain neuron-level attribution states, while SNKE additionally keeps target residuals, structural encodings, and mask-related tensors during editing. SNKE is therefore better understood as an accuracy-oriented editor with additional memory demands, rather than as the most lightweight editing method.

Table 5: Runtime and peak GPU memory usage of KGMET, NMKE, and SNKE on GPT-2-XL with 2,000 MultiCounterFact edits and an editing step size of 100.

Method	Total Time (s)	Chunk Time (s)	Edit Time (s)	Memory (MiB)
KGMET	8549.15	427.46	4.27	12066.64
NMKE	3382.49	169.12	1.69	38466.30
SNKE	7581.19	379.06	3.79	38734.65

5.6. Neuron Modification Analysis

The neuron modification analysis provides a mechanism-level inspection of the proposed mask design. NMKE uses a binary mask to decide which neurons should be updated, whereas SNKE separates cross-prompt consistency and prompt-specific peak response before applying the soft gate. This analysis examines whether SNKE changes the composition of edited neurons rather than simply increasing the number of updated neurons.

As shown in Table 6, NMKE selects 3198.40 neurons for updating, whereas SNKE selects 3351.84. The update scope is slightly broader but remains controlled. SNKE also identifies 2519.32 consistency neurons and 2513.52 peak-response neurons, showing that the selected set contains different response patterns. These counts do not by themselves prove that each neuron group improves performance, but they support the mechanism-level interpretation that SNKE edits a more structured neuron set. The stronger Gen. scores should therefore be interpreted together with the main results and the mask design, while the broader selection also helps explain why SNKE does not always achieve the best Loc.

Table 6: Neuron modification statistics under different mask designs. The table reports the total neurons, updated and non-updated neurons, consistency neurons, and peak-response neurons.

Method	Total Neurons	Mask = 1	Mask = 0	Consistency Neurons	Peak-response Neurons
KGMET	6400	-	-	-	-
NMKE	6400	3198.40	3201.60	-	-
SNKE	6400	3351.84	3048.16	2519.32	2513.52

5.7. Long-range Edit Capability

The long-range edit experiment tests whether editing methods remain effective as sequential updates accumulate. We use GPT-2-XL under the ZsRE sequential editing setting, with a maximum of 5,000 edits and an editing step size of 100. Fig 2 reports the main checkpoints at 1000, 2000, 3000, 4000, and 5000 edits.

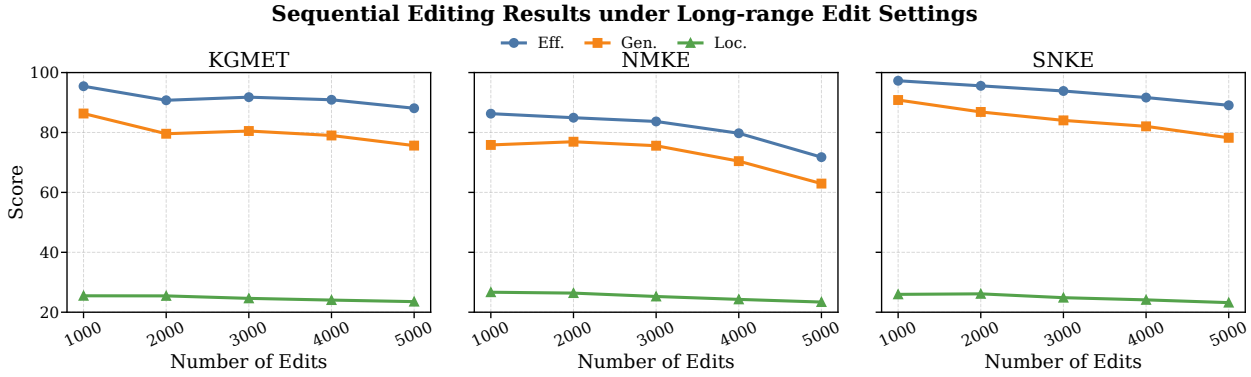


Figure 2: Long-range sequential editing trends on GPT-2-XL under the ZsRE setting. Each subplot reports Eff., Gen., and Loc. for KGMET, NMKE, and SNKE as the number of edits increases from 1,000 to 5,000.

SNKE keeps the highest Eff. and Gen. scores at all checkpoints, which suggests that target-direction attribution and soft gates slow the degradation caused by repeated updates. The scores still decline as the number of edits grows. This decline is expected: each edit adds a new parameter perturbation, and later edits can partially overwrite earlier ones. Loc. remains low for all methods because many locality prompts are semantically close to the edited facts. For SNKE, the same target-direction and structural signals that improve propagation may also expand the influence of an edit when neighboring facts share similar relational contexts. In this setting, preserving unrelated knowledge is harder than maintaining the edited answer itself.

5.8. Ablation Analysis

The ablation analysis evaluates which SNKE components contribute to editing performance under a large-scale factual editing setting. The experiment uses MultiCounterFact with 2,000 edits on LLaMA 3. We compare the original SNKE model with five variants that remove or isolate major components: target residual alignment, graph guidance, and neuron masking. In the neuron masking part, we have also implemented two masking methods: consistency-only masking and peak-response-only masking.

The ablation results in Table 7 show that SNKE obtains the highest Eff., Loc., and Consis. scores. This indicates that the full design improves target fact rewriting while better preserving locality and relation-aware generation consistency. Without graph guidance, SNKE increases Gen. by 0.03, but Consis. drops, which suggests that graph information mainly stabilizes semantic consistency rather than paraphrase transfer. When the neuron mask is removed, the model achieves the highest Flu. score, but its lower Consis. indicates that weaker neuron-level constraints increase surface diversity at the cost of relation-aware generation. The consistency-only and peak-response-only variants also perform below the full model. It shows that the two neuron groups are complementary. These results support combining target residual alignment, graph-guided attribution, and soft neuron masking for stable knowledge editing.

6. Limitation

SNKE still has practical limits. Its graph-aware attribution and residual-guided updates require extra memory, which may restrict use on larger backbones or denser edit batches. Multi-hop portability also remains difficult: SNKE improves the scores, but the absolute performance leaves substantial room for better reasoning-chain propagation. Finally, stronger propagation can reduce locality or fluency when the edited fact is close to neighboring facts, suggesting that future editors need finer control over where an edit should stop.

7. Conclusion and Future Work

Our paper introduces SNKE, a structure-guided neuron-level editor for large language models. The main finding is that responsive neurons are not always useful editing targets. Effective editing also requires checking whether their update directions support the target fact and whether the edited fact remains consistent with nearby relational structure. SNKE addresses this issue by combining target residual alignment, graph-aware attribution, and soft-gated updates. The experimental results show more stable factual updates across paraphrased prompts, multi-hop reasoning chains, and long edit sequences.

Future work will focus on reducing the memory cost of structural attribution and improving locality under dense sequential editing. Another direction is to extend direction-aware and structure-aware editing to temporal and multimodal knowledge, where facts change across time, text, and visual evidence.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, R. Garnett (Eds.), Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA, 2017, pp. 5998–6008.

Table 7: Ablation results on MultiCounterFact with 2,000 edits using LLaMA 3. Each variant removes or isolates one SNKE component to assess its contribution to editing performance. **Bold font** denotes the best results.

Model	Ablation Setting	Eff.↑	Gen.↑	Loc.↑	Flu.↑	Consis.↑
LLaMA 3	**SNKE**	**99.15**	94.25	**67.95**	602.06	**33.48**
	w/o Target Residual	99.10	93.98	67.57	626.88	28.94
	w/o Graph Guidance	99.05	**94.28**	67.58	625.29	31.23
	w/o Neuron Mask	99.00	93.90	67.92	**627.81**	27.77
	Consistency Only	99.00	93.75	67.54	615.34	26.11
	Peak-response Only	99.00	94.00	67.56	626.18	29.17

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual, 2020.

[3] F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, A. H. Miller, Language models as knowledge bases?, in: K. Inui, J. Jiang, V. Ng, X. Wan (Eds.), Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019, Association for Computational Linguistics, 2019, pp. 2463–2473. doi:10.18653/V1/D19-1250.

[4] A. Roberts, C. Raffel, N. Shazeer, How much knowledge can you pack into the parameters of a language model?, in: B. Webber, T. Cohn, Y. He, Y. Liu (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020, Association for Computational Linguistics, 2020, pp. 5418–5426. doi:10.18653/V1/2020.EMNLP-MAIN.437.

[5] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi, H. Wang, Continual learning of large language models: A comprehensive survey, ACM Comput. Surv. 58 (5) (2026) 120:1–120:42. doi:10.1145/3735633.

[6] Y. Li, Y. Zhou, J. Zhao, W. Du, R. Yao, B. Liu, Counterfactual knowledge maintenance for unsupervised domain adaptation, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025, ijcai.org, 2025, pp. 1476–1484. doi:10.24963/IJCAI.2025/165.

[7] N. D. Cao, W. Aziz, I. Titov, Editing factual knowledge in language models, in: M. Moens, X. Huang, L. Specia, S. W. Yih (Eds.), Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021, Association for Computational Linguistics, 2021, pp. 6491–6506. doi:10.18653/V1/2021.EMNLP-MAIN.522.

[8] E. Mitchell, C. Lin, A. Bosselut, C. Finn, C. D. Manning, Fast model editing at scale, in: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022, OpenReview.net, 2022.

[9] K. Meng, D. Bau, A. Andonian, Y. Belinkov, Locating and editing factual associations in GPT, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022.

[10] K. Meng, A. S. Sharma, A. J. Andonian, Y. Belinkov, D. Bau, Mass-editing memory in a transformer, in: The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023, OpenReview.net, 2023.

[11] E. Mitchell, C. Lin, A. Bosselut, C. Finn, C. D. Manning, Fast model editing at scale, in: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022, OpenReview.net, 2022.

[12] E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, C. Finn, Memory-based model editing at scale, in: K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvári, G. Niu, S. Sabato (Eds.), International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA,

Proceedings of Machine Learning Research, PMLR, 2022, pp. 15817–15831.

[13] J. Liu, J. Sun, S. Shen, C. Yang, S. Wang, Edit less, achieve more: Dynamic sparse neuron masking for lifelong knowledge editing in llms, CoRR abs/2510.22139 (2025). arXiv:2510.22139, doi:10.48550/ARXIV.2510.22139.

[14] C. Hsueh, P. K. Huang, T. Lin, C. Liao, H. Fang, C. Huang, Y. Chen, Editing the mind of giants: An in-depth exploration of pitfalls of knowledge editing in large language models, in: Y. Al-Onaizan, M. Bansal, Y. Chen (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024, Findings of ACL, Association for Computational Linguistics, 2024, pp. 9417–9429. doi:10.18653/V1/2024.FINDINGS-EMNLP.550.

[15] G. He, X. Song, F. Wang, A. Sun, Benchmarking and rethinking knowledge editing for large language models, CoRR abs/2505.18690 (2025). arXiv:2505.18690, doi:10.48550/ARXIV.2505.18690.

[16] N. Zhang, Y. Yao, B. Tian, P. Wang, S. Deng, M. Wang, Z. Xi, S. Mao, J. Zhang, Y. Ni, S. Cheng, Z. Xu, X. Xu, J. Gu, Y. Jiang, P. Xie, F. Huang, L. Liang, Z. Zhang, X. Zhu, J. Zhou, H. Chen, A comprehensive study of knowledge editing for large language models, CoRR abs/2401.01286 (2024). arXiv:2401.01286, doi:10.48550/ARXIV.2401.01286.

[17] S. Wang, Y. Zhu, H. Liu, Z. Zheng, C. Chen, J. Li, Knowledge editing for large language models: A survey, ACM Comput. Surv. 57 (3) (2025) 59:1–59:37. doi:10.1145/3698590.

[18] Y. Jiang, Y. Wang, C. Wu, W. Zhong, X. Zeng, J. Gao, L. Li, X. Jiang, L. Shang, R. Tang, Q. Liu, W. Wang, Learning to edit: Aligning llms with knowledge editing, in: L. Ku, A. Martins, V. Srikumar (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, Association for Computational Linguistics, 2024, pp. 4689–4705. doi:10.18653/V1/2024.ACL-LONG.258.

[19] Z. Yu, S. Ananiadou, Neuron-level knowledge attribution in large language models, in: Y. Al-Onaizan, M. Bansal, Y. Chen (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024, Association for Computational Linguistics, 2024, pp. 3267–3280. doi:10.18653/V1/2024.EMNLP-MAIN.191.

[20] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, F. Wei, Knowledge neurons in pretrained transformers, in: S. Muresan, P. Nakov, A. Villavicencio (Eds.), Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022, Association for Computational Linguistics, 2022, pp. 8493–8502. doi:10.18653/V1/2022.ACL-LONG.581.

[21] X. Zhang, Y. Liang, F. Meng, S. Zhang, Y. Chen, J. Xu, J. Zhou, Multilingual knowledge editing with language-agnostic factual neurons, in: O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, S. Schockaert (Eds.), Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025, Association for Computational Linguistics, 2025, pp. 5775–5788.

[22] J. Xie, P. Cao, Y. Chen, Y. Chen, K. Liu, J. Zhao, MEMLA: enhancing multilingual knowledge editing with neuron-masked low-rank adaptation, CoRR abs/2406.11566 (2024). arXiv:2406.11566, doi:10.48550/ARXIV.2406.11566.

[23] X. Wang, Z. Chen, H. Wang, L. H. U, Z. Li, W. Guo, Large language model enhanced knowledge representation learning: A survey, Data Sci. Eng. 10 (3) (2025) 315–338. doi:10.1007/S41019-025-00285-Y.

[24] B. Liu, X. Li, Large language models for knowledge graph embedding techniques, methods, and challenges: A survey, CoRR abs/2501.07766 (2025). arXiv:2501.07766, doi:10.48550/ARXIV.2501.07766.

[25] N. Ibrahim, S. AboulEla, A. F. Ibrahim, R. F. Kashef, A survey on augmenting knowledge graphs (kgs) with large language models (llms): models, evaluation metrics, benchmarks, and challenges, Discov. Artif. Intell. 4 (1) (2024) 76. doi:10.1007/S44163-024-00175-8.

[26] W. Yang, L. Some, M. Bain, B. H. Kang, A comprehensive survey on integrating large language models with knowledge-based methods, Knowl. Based Syst. 318 (2025) 113503. doi:10.1016/J.KNOSYS.2025.113503.

[27] A. Bordes, N. Usunier, A. García-Durán, J. Weston, O. Yakhnenko, Translating embeddings for modeling multi-relational data, in: C. J. C. Burges, L. Bottou, Z. Ghahramani, K. Q. Weinberger (Eds.), Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States, 2013, pp. 2787–2795.

[28] Z. Sun, Z. Deng, J. Nie, J. Tang, Rotate: Knowledge graph embedding by relational rotation in complex space, in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019, OpenReview.net, 2019.

[29] M. S. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with

graph convolutional networks, in: A. Gangemi, R. Navigli, M. Vidal, P. Hitzler, R. Troncy, L. Hollink, A. Tordai, M. Alam (Eds.), The Semantic Web - 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3-7, 2018, Proceedings, Lecture Notes in Computer Science, Springer, 2018, pp. 593–607. doi:10.1007/978-3-319-93417-4_38.

[30] S. Cheng, N. Zhang, B. Tian, X. Chen, Q. Liu, H. Chen, Editing language model-based knowledge graph embeddings, in: M. J. Wooldridge, J. G. Dy, S. Natarajan (Eds.), Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada, AAAI Press, 2024, pp. 17835–17843. doi:10.1609/AAAI.V38I16.29737.

[31] M. Zhang, X. Ye, Q. Liu, P. Ren, S. Wu, Z. Chen, Knowledge graph enhanced large language model editing, in: Y. Al-Onaizan, M. Bansal, Y. Chen (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024, Association for Computational Linguistics, 2024, pp. 22647–22662. doi:10.18653/V1/2024.EMNLP-MAIN.1261.

[32] J. Fu, K. Wang, C. Guo, J. Fang, W. Zhang, S. Su, Knowledge graph-driven memory editing with directional interventions, in: C. Christodoulopoulos, T. Chakraborty, C. Rose, V. Peng (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025, Association for Computational Linguistics, 2025, pp. 4860–4874.

[33] L. Deng, Y. Han, L. Zhang, Y. Du, B. Li, Consistency-aware parameter-preserving knowledge editing framework for multi-hop question answering, CoRR abs/2509.18655 (2025). arXiv:2509.18655, doi:10.48550/ARXIV.2509.18655.

[34] Y. K. Atri, A. M. Alaa, T. Hartvigsen, Lifelong model editing with graph-based external memory, in: W. Che, J. Nabende, E. Shutova, M. T. Pilehvar (Eds.), Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, Findings of ACL, Association for Computational Linguistics, 2025, pp. 13336–13352.

[35] L. Yuan, Q. Huang, B. Zhu, Y. Cai, Q. Huang, C. Zheng, Z. Deng, T. Wang, Hybrid-dmkg: A hybrid reasoning framework over dynamic multimodal knowledge graphs for multimodal multihop QA with knowledge editing, in: S. Koenig, C. Jenkins, M. E. Taylor (Eds.), Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026, AAAI Press, 2026, pp. 28032–28040. doi:10.1609/AAAI.V40I33.40028.

[36] O. Levy, M. Seo, E. Choi, L. Zettlemoyer, Zero-shot relation extraction via reading comprehension, in: R. Levy, L. Specia (Eds.), Proceedings of the 21st Conference on Computational Natural Language Learning (CoNLL 2017), Vancouver, Canada, August 3-4, 2017, Association for Computational Linguistics, 2017, pp. 333–342. doi:10.18653/V1/K17-1034.

[37] Z. Zhong, Z. Wu, C. D. Manning, C. Potts, D. Chen, Mquake: Assessing knowledge editing in language models via multi-hop questions, in: H. Bouamor, J. Pino, K. Bali (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023, Association for Computational Linguistics, 2023, pp. 15686–15702. doi:10.18653/V1/2023.EMNLP-MAIN.971.

[38] L. Team, The llama 3 herd of models, CoRR abs/2407.21783 (2024). arXiv:2407.21783, doi:10.48550/ARXIV.2407.21783.

[39] P. Wang, N. Zhang, B. Tian, Z. Xi, Y. Yao, Z. Xu, M. Wang, S. Mao, X. Wang, S. Cheng, K. Liu, Y. Ni, G. Zheng, H. Chen, Easyedit: An easy-to-use knowledge editing framework for large language models, in: Y. Cao, Y. Feng, D. Xiong (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), ACL 2024, Bangkok, Thailand, August 11-16, 2024, Association for Computational Linguistics, 2024, pp. 82–93. doi:10.18653/V1/2024.ACL-DEMOS.9.

[40] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, Lora: Low-rank adaptation of large language models, in: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022, OpenReview.net, 2022.
URL https://openreview.net/forum?id=nZeVKeeFYf9

[41] J. Ma, H. Wang, H. Xu, Z. Ling, J. Gu, Perturbation-restrained sequential model editing, in: The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025, OpenReview.net, 2025.

[42] J. Gu, H. Xu, J. Ma, P. Lu, Z. Ling, K. Chang, N. Peng, Model editing harms general abilities of large language models: Regularization to the rescue, in: Y. Al-Onaizan, M. Bansal, Y. Chen (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024, Association for Computational Linguistics, 2024, pp. 16801–16819. doi:10.18653/V1/2024.EMNLP-MAIN.934.

[43] M. Zhang, X. Ye, Q. Liu, P. Ren, S. Wu, Z. Chen, Knowledge graph enhanced large language model editing, in: Y. Al-Onaizan, M. Bansal, Y. Chen (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024, Association for Computational Linguistics, 2024, pp. 22647–22662. doi:10.18653/V1/2024.EMNLP-MAIN.1261.